Техническое задание для разработчика: Парсер логов PsychoPy для анализа первого уровня в SPM25

1. Общая цель

Разработать скрипт (на Python или MATLAB), который принимает на вход файл логов PsychoPy (в формате текстового дампа словарей/структуры записей, например, `101_stropExp_kaz1_..._readable_2.txt`) и формирует на выходе структурированные файлы таймингов (обычно файлы `.mat` или текстовые `.txt` / `.csv` с колонками `onsets`, `durations`, `names`), необходимые для модуля **fMRI Model Specification** в SPM25.

2. Структура входных данных (PsychoPy Logs)

Файл состоит из метаданных эксперимента и последовательности записей (`RECORD`), представляющих собой словари Python.

- **Ключевые блоки (`Block`):**
    - Практикующие блоки: `PB_S` (Standard Practice), `PB_C` (Concept Practice). *Обычно исключаются из сканирования первого уровня.*
    - Экспериментальные сессии/раунды: `Block1_ColorWord` (RUN 1) и `Block2_ConceptWord` (RUN 2). Именно они интересуют для fMRI анализа.
- **Важные поля внутри каждой записи trial (например, `RECORD 17` и далее):**
    - `Block`: название блока (`Block1_ColorWord`, `Block2_ConceptWord`).
    - `Language`: язык стимула (`English`, `Kazakh`, `Russian`).
    - `congruent`: флаг конгруэнтности (`1` — конгруэнтный, `0` — неконгруэнтный).
    - `transition`: тип переключения языков (`filler`, `repeat`, `switch`).
    - `switchDirection`: направление переключения (например, `English->Kazakh`, `Kazakh->Russian`, `None`).
    - `MixS.started` / `mixC.started` (или поле `thisRow.t`): **абсолютное время начала предъявления стимула (onset)** в секундах от старта эксперимента.

- MixS.stopped / mixC.stopped (или поля длительности): момент окончания стимула.
- luminaResp / practiceResp: ответ испытуемого.
- luminaCorrect / practiceCorrect: правильность ответа (1 — верно, 0 — ошибка, None — нет ответа / пропуск).
- luminaRT / practiceRT: время реакции (Reaction Time).

3. Требования к логике парсера

Шаг 1: Фильтрация и разделение по сессиям (Runs)

1. Просканировать файл построчно, извлечь метаданные (participant, expName, frameRate и др.).
2. Отфильтровать технические записи (Other routine, WaitScanner, InitialFix, End_Block1 и т.д.) и тренировочные блоки (PB_S, PB_C).
3. Разделить экспериментальные испытания по сессиям на основе поля Block:
   - **Сессия 1:** Block1_ColorWord (соответствует RUN 1 fMRI).
   - **Сессия 2:** Block2_ConceptWord (соответствует RUN 2 fMRI).

Шаг 2: Выделение условий (Conditions) для Design Matrix

Для каждого раунда парсер должен сформировать регрессоры (условия) общего линейного моделирования (GLM). В зависимости от исследовательского дизайна, условия могут группироваться по:

- **Фактору языка и конгруэнтности** (например: Eng_Congruent, Eng_Incongruent, Kaz_Congruent, Kaz_Incongruent, Rus_Congruent, Rus_Incongruent).
- **Фактору переключения языков (Language Switch)**: Switch, Repeat, Filler.
- **Учет правильности ответов (опционально для удаления ошибок):** Испытания с luminaCorrect == 0 или None могут выноситься в отдельный регрессор ошибок (error trials) или исключаться, а правильные — идти в основные условия.

Шаг 3: Сбор параметров для каждого события

Для каждого выявленного условия скрипт должен собрать три вектора (формат SPM):

1. names: Название условия (строка).
2. onsets: Список временных меток (onset в секундах). В качестве базового времени брать MixS.started (или thisRow.t). *Важно:* Время должно быть пересчитано относительно начала конкретной сессии (путем вычитания

времени старта первого триала сессии или триала InitialFix), если SPM требует тайминги сессии, а не глобальные часы эксперимента.

3. durations: Длительность стимула. Обычно для коротких предъявлений задается 0 (событийный дизайн / event-related) или рассчитывается как разница stopped - started.

4. Формат выходных данных

Парсер должен сохранять результат в файл, понятный для SPM25 (например, .mat-файл Matlab со структурой names, onsets, durations для каждой сессии отдельно, либо текстовые файлы, импортируемые пакетно).

Пример структуры сохраняемого .mat файла для одной сессии:

```python
# Пример структуры на выходе Python-скрипта (для scipy.io.savemat)
session_data = {
    'names': ['Eng_Congruent', 'Eng_Incongruent', 'Kaz_Congruent', 'Kaz_Incongruent'
    'onsets': [[10.5, 25.2, ...], [15.1, 30.4, ...], [...], [...]],
    'durations': [[0, 0, ...], [0, 0, ...], [...], [...]]
}
```

5. Дополнительные требования к скрипту

- **Параметрические модуляции (Parametric Modulators):** Предусмотреть возможность добавления времени реакции (luminaRT) в качестве параметрического модулятора для каждого условия.

- **Обработка пропусков:** Корректно обрабатывать строки со значениями None (когда испытуемый не дал ответ), чтобы скрипт не падавший с ошибкой TypeError.

- **Логирование работы:** Вывод в консоль отчета по итогам парсинга (например: *«Успешно обработан испытуемый 101: Сессия 1 — 120 триалов, Сессия 2 — 45 триалов. Ошибок не обнаружено»*).